

Capítulo 09

Precision

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 31/07/2026

Capítulo 09 — Precision (treinar com menos bits)

Objetivo de aprendizagem: entender o que são `fp32`, `fp16` e `bf16` no nível dos bits, por que treinar em 16 bits quebra o modelo de duas formas distintas (*overflow* e *underflow*), e como a **precisão mista** e o **loss scaling** resolvem isso. Vamos medir tudo — inclusive descobrir que, neste hardware, metade da promessa não se cumpre.

Pré-requisitos: Capítulos 1-8. Em especial o Capítulo 8 (dispositivos) e a metodologia de medição que estabelecemos lá.

Arquivos: - `floats.py` — anatomia de um float: bits, alcance, precisão, limites - `precision_bench.py` — velocidade e memória por precisão - `loss_scaling.py` — o problema medido e o **GradScaler do zero** - `device.py` — detecção de dispositivo (do Capítulo 8) - `exercicios.md` — exercícios

1. Por que mexer nos bits

Duas razões, e é importante separá-las porque **elas não vêm juntas**:

1. **Memória.** Um número de 16 bits ocupa metade de um de 32. Isso permite modelo maior, batch maior ou contexto mais longo na mesma placa.
2. **Velocidade.** Se o hardware tiver unidades dedicadas a 16 bits (como os *tensor cores* da NVIDIA), a mesma matmul roda várias vezes mais rápido.

O benefício 1 é garantido pela aritmética. O benefício 2 **depende do hardware e do backend** — e a Seção 6 mostra um caso real em que ele simplesmente não aparece.

Em treino de LLM, memória costuma ser o recurso mais escasso. Um modelo de 1 bilhão de parâmetros em `fp32` ocupa 4 GB só de pesos — e, com AdamW, mais uns 12 GB de gradientes e momentos (`m` e `v`), antes de qualquer ativação. Cortar isso pela metade não é um luxo de otimização; é o que decide se o treino cabe na máquina.

2. Anatomia de um float

Um número de ponto flutuante tem três partes:

```
[sinal | expoente | mantissa]

sinal : 1 bit — positivo ou negativo
expoente : a ESCALA (ordem de grandeza) -> mais bits = mais ALCANCE
mantissa : os DÍGITOS significativos -> mais bits = mais PRECISÃO
```

Os três formatos que importam:

Formato	Sinal	Expoente	Mantissa	Total
fp32 (float)	1	8	23	32 bits
fp16 (half)	1	5	10	16 bits
bf16 (bfloat16)	1	8	7	16 bits

Repare no detalhe que decide tudo: o **bf16 tem os mesmos 8 bits de expoente do fp32**. Ele sacrifica precisão para manter o alcance. O fp16 faz o contrário — guarda mais dígitos, mas num intervalo estreito.

Rodando `python floats.py`, os bits de **3.14159**:

```
fp32: 0 10000000 10010010000111111010000
fp16: 0 10000 1001001000
bf16: 0 10000000 1001001
```

O expoente do bf16 é **idêntico** ao do fp32 (**10000000**); a mantissa é o pedaço truncado.

3. Alcance e precisão, medidos

```
formato maior menor normal epsilon
fp32 3.403e+38 1.175e-38 1.192e-07
fp16 6.550e+04 6.104e-05 9.766e-04
bf16 3.390e+38 1.175e-38 7.812e-03
```

Três leituras:

- O **fp16 estoura em 65.504**. Qualquer valor maior vira **inf**.
- O **bf16 alcança 3,4e38** — o mesmo do fp32, como esperado pelos 8 bits de expoente.
- Em troca, o **epsilon do bf16 é 8x pior**: 7,8e-03 contra 9,8e-04.

O *epsilon* é a menor diferença relativa que o formato distingue. Um epsilon de 7,8e-03 significa cerca de **2 casas decimais significativas** — parece pouquíssimo para treinar uma rede. E ainda assim o bf16 é o formato preferido. A Seção 5 explica por quê.

Um cuidado ao medir precisão: comparando um único valor, fp16 e bf16 podem dar exatamente o mesmo erro (basta que os bits extras do fp16 sejam zeros). Por isso o `floats.py` mede vários valores e reporta a **média**: erro relativo médio de **1,54e-04** (fp16) contra **8,67e-04** (bf16) — o bf16 é ~6x pior, coerente com os 3 bits menos de mantissa ($2^3 = 8x$).

4. Onde 16 bits quebram

Overflow: números grandes demais

```
1e+04 -> fp16 10000.0 | bf16 9.984e+03
6e+04 -> fp16 60000.0 | bf16 5.990e+04
7e+04 -> fp16 inf | bf16 7.014e+04
1e+05 -> fp16 inf | bf16 9.984e+04
```

Passando de 65.504, o fp16 devolve **inf**. Ativações e valores intermediários da loss alcançam essa faixa com facilidade em redes grandes.

Underflow: números pequenos demais

```
1e-04 -> fp16 1.000e-04 | bf16 1.001e-04
1e-06 -> fp16 1.013e-06 | bf16 9.984e-07
1e-08 -> fp16 0.000e+00 | bf16 1.001e-08 <- virou ZERO
1e-10 -> fp16 0.000e+00 | bf16 1.000e-10 <- virou ZERO
```

Este é o mais perigoso dos dois, e a razão é sutil: **um gradiente que vira zero não atualiza o peso — e o treino trava sem dar erro nenhum**. Não há exceção, não há **NaN** na loss. O modelo simplesmente aprende menos, e você não sabe por quê.

5. Por que o bf16 ganhou

Gradientes típicos de uma rede treinando ficam entre 1e-4 e 1e-8. Cruzando isso com os limites de cada formato:

Gradiente	fp16	bf16
1e-03	1,000e-03	9,995e-04
1e-05	1,001e-05	1,001e-05
1e-07	1,192e-07	1,001e-07
1e-08	0 (zero)	1,001e-08
1e-09	0 (zero)	9,968e-10

Aqui está a resposta inteira:

Para treinar, alcance importa mais que precisão. A direção aproximada do gradiente é útil; um gradiente zerado não é. O bf16 troca dígitos significativos — que o gradient descent tolera bem, porque ele dá milhares de passos pequenos — por alcance, que é o que impede o número de desaparecer.

É por isso que TPUs, A100 e H100 adotaram o bf16 como formato padrão de treino, e por que o fp16 precisa de uma engrenagem extra (Seção 7).

6. O que medimos de verdade — e o que não se cumpriu

Aqui o capítulo entrega um resultado que contraria o discurso comum. Multiplicando matrizes 1024×1024:

Dispositivo	fp32	fp16	bf16	Ganho vs fp32
CPU	5,35 ms	1.951 ms	2.401 ms	~0,003x (centenas de vezes pior!)
Radeon RX 7600 (DirectML)	0,40 ms	0,41 ms	n/d	0,98x (nenhum)

Na CPU, 16 bits é catastróficamente mais lento. Não é erro de medição: CPUs não têm unidades de aritmética de 16 bits para matmul, e o PyTorch emula a operação convertendo elemento por elemento. O resultado é da ordem de **350 a 450 vezes** pior que o **fp32**.

Sobre a imprecisão desse número: esse caso patológico tem variância alta — entre execuções eu medi de 1.950 a 4.017 ms. Não vale perseguir o valor exato; o que importa é a ordem de grandeza, e ela é inequívoca. Já os números da GPU são estáveis e reprodutíveis (0,98-0,99x em todas as execuções).

Na Radeon via DirectML, o ganho é zero. O **fp16** roda exatamente na mesma velocidade que o **fp32** (0,98x). O hardware RDNA3 tem, em teoria, aritmética de 16 bits em taxa dobrada, mas o DirectML não despacha para esses kernels.

E o **bf16** **não existe** nesta placa — pior, ele **aborta o processo**:

```
[F730 10:46:20.000000000 dml_util.cc:118] Invalid or unsupported data type BFloat16.
```

O **[F]** é *fatal*: não é uma exceção Python que se possa capturar com **try/except**, é o processo morrendo. Por isso o **precision_bench.py** consulta uma lista de suporte conhecido **antes** de tentar, em vez de tentar e tratar o erro:

```
def suportado(device, dtype):
    if device.type == "privateuseone" and dtype == torch.bfloat16:
        return False, "DirectML aborta o processo com bfloat16"
    return True, ""
```

A conclusão honesta: neste hardware, precisão reduzida compra **memória, não velocidade**. O benefício de velocidade exige hardware e software preparados — na prática, NVIDIA com CUDA e tensor cores. Se você mediu ganho zero na sua máquina, não é porque a técnica não funciona: é porque o seu caminho de execução não a implementa.

O benefício de memória, esse sim, é garantido:

Formato	Bytes/parâmetro	Modelo de 1B parâmetros (só pesos)
---------	-----------------	------------------------------------

fp32	4	4,0 GB
fp16	2	2,0 GB
bf16	2	2,0 GB

7. Loss scaling: o truque que salva o fp16

O problema medido em `loss_scaling.py`, com uma rede pequena de pesos pequenos:

```
total de gradientes: 26896
zerados em fp32: 0 ( 0.0%)
zerados em fp16: 13541 ( 50.3%) <- perdidos por underflow

magnitude mediana dos gradientes (fp32): 2.93e-08
menor valor normal do fp16: 6.10e-05
```

Metade dos gradientes vira zero. Metade do modelo, efetivamente, não aprende.

A solução se apoia numa propriedade elementar da derivada — ela é **linear**:

$$d(k \cdot L)/dw = k \cdot dL/dw$$

Ou seja: se multiplicarmos a **loss** por um fator **k** grande antes do **backward**, **todos** os gradientes saem multiplicados por **k**, longe da zona de underflow. Depois dividimos por **k** antes de atualizar os pesos. O resultado final é o mesmo do fp32 — só os números intermediários passaram a ser representáveis.

```
loss × 1024 -> backward -> gradientes × 1024 -> ÷ 1024 -> usar
```

Funciona? Medindo:

Escala	Zerados	%	infs	Situação
1	13.541	50,3%	0	perde muito gradiente
16	1.026	3,8%	0	perde muito gradiente
256	68	0,3%	0	ainda perde um pouco
1.024	14	0,1%	0	ok
65.536	0	0,0%	0	ok
4.194.304	0	0,0%	26.640	overflow

67.108.864	0	0,0%	26.896	overflow
------------	---	------	--------	----------

Existe uma **janela**: escala pequena não resolve o underflow, escala grande causa overflow. E — o ponto crucial — **a janela se move durante o treino**, porque a magnitude dos gradientes muda conforme o modelo aprende.

O GradScaler dinâmico

Por isso ninguém usa escala fixa. A solução é um controle de realimentação:

```
def passo(self, parametros):
    achou_inf = any(not torch.isfinite(p.grad).all() for p in parametros ...)
    if achou_inf:
        self.escala = max(1.0, self.escala / self.fator) # recua
        self.descartados += 1
        return False # DESCARTA o passo
    for p in parametros:
        p.grad /= self.escala # desescala
    self.bons_seguidos += 1
    if self.bons_seguidos >= self.intervalo:
        self.escala *= self.fator # ousa mais
    return True
```

A lógica: **apareceu inf, recua e joga o passo fora; passaram N passos limpos, arrisca uma escala maior**. O algoritmo persegue continuamente a maior escala que ainda não estoura.

Rodando a simulação (com gradientes que crescem de propósito):

```
passo escala aplicado?
  5 131072 sim
 10 262144 sim
 15 524288 sim
 20 1048576 sim
 21 524288 DESCARTADO
 25 524288 sim

passos descartados: 1/25
```

Ele sobe até 1.048.576, estoura, recua para 524.288 e segue. **Descartar passos é aceitável** — com a escala bem ajustada isso acontece em menos de 1% das iterações.

Na prática você usa `torch.amp.GradScaler`, que implementa exatamente este algoritmo. Mas agora você sabe o que ele faz e por quê.

8. Precisão mista e os pesos mestres

"Precisão mista" **não** significa converter o modelo todo para 16 bits. Significa escolher, operação por operação:

Em 16 bits (rápido, e o erro não acumula)

Em 32 bits (onde a precisão importa)

matmul e convolução — onde está ~95% do tempo	a cópia mestra dos pesos
—	somas longas: softmax, LayerNorm, loss
—	o estado do otimizador (m e v)

O ganho vem da coluna da esquerda; a estabilidade, da direita.

Por que os pesos mestres ficam em fp32

Este é o ponto menos intuitivo, e o `precision_bench.py` demonstra. Somando uma atualização típica de $1e-4$ a um peso de valor **1,0**, cem vezes:

```
peso = 1.0, atualizacao = 1e-04, aplicada 100 vezes:
fp32: 1.010002 (esperado 1.010000, erro 1.66e-06)
fp16: 1.000000 (esperado 1.010000, erro 1.00e-02)
bf16: 1.000000 (esperado 1.010000, erro 1.00e-02)
```

Leia com atenção: em 16 bits o peso **não se moveu nem um pouco** depois de cem atualizações. Não é um erro de arredondamento acumulado — é que cada soma individual foi arredondada de volta para **1.0**, todas as cem vezes.

Em 16 bits, uma atualização pequena somada a um peso de ordem 1 cai **abaixo do epsilon do formato** e é arredondada para fora — o peso simplesmente não muda. Repetido por milhares de passos, o treino estagna.

É por isso que a economia de memória real fica **abaixo** dos 50% que a aritmética ingênua sugere: você mantém uma cópia fp32 dos pesos além da cópia de trabalho em 16 bits.

9. Como se escreve na prática

Com fp16 — precisa de scaler:

```
scaler = torch.amp.GradScaler(device="cuda")
for x, y in dados:
    with torch.autocast(device_type="cuda", dtype=torch.float16):
        loss = F.cross_entropy(modelo(x), y) # matmuls em fp16
    opt.zero_grad(set_to_none=True)
    scaler.scale(loss).backward() # escala o backward
    scaler.step(opt) # desescala e aplica
    scaler.update() # ajusta a escala
```

Com bf16 — não precisa de scaler:

```
for x, y in dados:
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        loss = F.cross_entropy(modelo(x), y)
    opt.zero_grad(set_to_none=True)
    loss.backward()
```

```
opt.step()
```

A segunda versão é mais simples, e é por isso que o bf16 virou padrão onde o hardware o suporta: **ele elimina uma engrenagem inteira do treino.**

10. Os limites deste capítulo, declarados

Sendo explícito sobre o que foi verificado e o que não pôde ser:

Dispositivo	fp16	bf16	autocast
CPU	[OK] funciona (mas centenas de vezes mais lento)	[OK] funciona	[OK] funciona
Radeon RX 7600 (DirectML)	[OK] funciona, ganho 0,98x	[nao] aborta o processo	[nao] não implementado
NVIDIA + CUDA	não testado aqui	não testado aqui	não testado aqui

O que **foi medido**: a anatomia dos formatos, os limites de alcance e precisão, o underflow de gradientes (50,3%), a janela do loss scaling, o comportamento do scaler dinâmico, a estagnação dos pesos em 16 bits, e a velocidade em CPU e nesta GPU.

O que **não foi medido**: o ganho de velocidade em tensor cores, o treino completo em bf16 na GPU, e o **fp8** (que exige hardware classe H100). Onde eu não medi, não afirmo.

11. Resumo do capítulo

- Um float é [sinal | expoente | mantissa]. **Expoente = alcance; mantissa = precisão.**
- **fp16**: 5 bits de expoente -> estoura em 65.504 e zera abaixo de $\sim 6e-5$. **bf16**: 8 bits (como o fp32) -> mesmo alcance, mas epsilon 8x pior.
- Duas falhas distintas: **overflow** (vira **inf**) e **underflow** (vira zero). O segundo é pior porque é **silencioso** — o treino trava sem erro.
- **Para treinar, alcance > precisão.** Por isso o bf16 é o padrão do hardware moderno.
- **Loss scaling**: multiplica a loss por **k**, gradientes saem $\times k$, divide antes de atualizar. Existe uma **janela** de escalas seguras, e ela se move -> use um scaler **dinâmico** (implementamos um do zero).
- **Precisão mista** = 16 bits nas matmuls, 32 bits nos pesos mestres, somas longas e estado do otimizador.
- **Medido**: 50,3% dos gradientes zeram em fp16; o loss scaling recupera 100% deles.
- **Medido, e contra o discurso comum**: neste hardware, 16 bits **não acelera nada** (CPU centenas de vezes mais lenta; DirectML 0,98x). O ganho de memória é real; o de velocidade depende de

hardware e backend.

O que vem no Capítulo 10

Fecha a trilogia "Need for Speed": no **Capítulo 10 — Distributed** vamos treinar em **vários dispositivos ao mesmo tempo**, entendendo o **all-reduce** que sincroniza os gradientes. Como esta máquina tem uma GPU só, verificaremos a **mecânica** com múltiplos processos na CPU — e o limite será declarado, como aqui.

-> Antes de seguir, faça os **exercícios**.

Exercícios — Capítulo 09 (Precision)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Os exercícios E1 a E5 rodam **na CPU** e não precisam de GPU. O E6 e o E7 precisam de GPU para a parte de velocidade.

E1 — Contas com bits (aquecimento)

Sem rodar código: 1. O bf16 tem 8 bits de expoente e 7 de mantissa; o fp16 tem 5 e 10. Qual dos dois representa o número `100000`? E qual representa `0,000000001` (1e-9)? 2. Quantos valores distintos um formato com 7 bits de mantissa consegue representar **entre 1 e 2**? E com 10 bits? (Dica: é `2^bits`.) 3. Por que dobrar os bits de mantissa **não** dobra a precisão, mas a multiplica por muito mais?

E2 — Ache o limite na mão

Escreva um laço que, começando em 1,0, multiplique por 2 até que o valor vire `inf` em fp16. 1. Em que potência de 2 isso acontece? Confere com `torch.info(torch.float16).max`? 2. Faça o mesmo dividindo por 2 até chegar a zero. Em que potência? Compare com `.tiny` — por que dá para ir **abaixo** do `.tiny` antes de zerar? (Pesquise "números subnormais".) 3. Repita para bf16 e explique a diferença.

E3 — O epsilon na prática

1. Em fp16, calcule `1.0 + x` para `x = 1e-2, 1e-3, 1e-4`. A partir de qual valor a soma deixa de mudar o resultado?
2. Repita em bf16. O limite é maior ou menor? Por quê?
3. Relacione com a Seção 4 da apostila: por que os **pesos mestres** precisam ficar em fp32?

E4 — A janela do loss scaling

No `loss_scaling.py`, mexa no `modelinho` para que os gradientes fiquem **maiores** (aumente o `p.mul_(0.02)` para `p.mul_(0.5)`). 1. A porcentagem de gradientes zerados em fp16 muda? 2. A janela de escalas seguras se move? Para cima ou para baixo? 3. Isso justifica o scaler **dinâmico**? Explique com os seus números.

E5 — Melhore o GradScaler

O `GradScalerSimples` da apostila reduz a escala pela metade a cada overflow e a dobra a cada N passos bons. 1. Qual o problema de um `fator` muito grande (ex.: 16)? E muito pequeno (ex.: 1,01)? 2. Implemente uma variante que, ao detectar overflow, **não** reduza abaixo de um piso configurável. Por que um piso é útil? 3. O `torch.amp.GradScaler` real usa `growth_interval=2000` por padrão, não 100.

Por que um intervalo tão longo?

E6 — Seu hardware acelera 16 bits? (precisa de GPU)

Rode `precision_bench.py` na sua máquina. 1. O fp16 é mais rápido que o fp32 na sua GPU? Quanto? 2. **Cuidado com a conclusão:** na máquina desta apostila o ganho foi **nulo** (0,99x). Se o seu também for, isso significa que precisão reduzida é inútil? (Dica: releia a Seção 6 — há dois benefícios distintos.) 3. Se você tem NVIDIA: procure "tensor cores" na especificação da sua placa. O ganho que você mediu é compatível com o prometido?

E7 — Treine de verdade em precisão mista (desafio)

Pegue o `transformer.py` do Capítulo 5 e adapte para usar `torch.autocast`. 1. Com `bfloat16` (sem scaler): a loss final fica igual à do fp32? Compare com os números do Capítulo 5 (validação 1,811). 2. Com `float16` **sem** scaler: o treino degrada? Meça. 3. Com `float16` **com** `torch.amp.GradScaler`: recupera a qualidade? 4. Meça o **pico de memória** nos três casos (use `torch.cuda.max_memory_allocated()` se tiver NVIDIA). A economia chega aos 50% teóricos? Por que não?